

RTL Design Sherpa

FPGA Systems Host Layer Specification 1.0

September 30, 2026

Table of Contents

1 Fpga Systems Mas Index....10

2 Overview....11

2.1 What this book covers....11

2.2 What this book does not cover....11

2.3 The one property worth protecting....12

3 Layering....13

3.0.1 Figure 1.1: Layering of the host-side stack....13

3.1 Entry point....13

3.2 Area layer....14

3.3 Shared layer....14

3.4 Transport and silicon....14

3.5 The direction rule....14

4 Acronyms and terminology....15

4.1 Terms used precisely in this book....16

5 References....17

5.1 In this repository....17

5.2 Tracker....17

5.3 External....18

5.4 A note on register documentation....18

6 Port discovery....19

6.0.1 Figure 2.1: The UART flow, host to silicon....19

6.1 The problem....19

6.2 What identifies a board....19
6.3 Matching is prefix-tolerant in both directions....20
6.4 The Genesys 2 exception....20
6.5 Listing what is attached....20
7 Probes....22
7.1 register_probe – read-only....22
7.2 scratch_probe – round-trips a scratch register....22
7.3 The resolution order....23
8 The link and the bridge....24
8.1 UartLink....24
8.2 UARTAxiBridge....24
8.3 The injection point....25
8.4 Resolving the bridge's location....25
9 The wire protocol....26
9.1 Write....26
9.2 Read....26
9.3 Defaults....26
9.4 A read of None is ambiguous, and that is worth knowing....27
9.5 Addressing....27
10 Sim and silicon equivalence....28
10.1 The mechanism....28
10.2 What differs, and what must not....28
10.3 The constraint that follows....29
10.4 The anti-pattern....29

11 The sequence model....30
11.1 Two rules, enforced rather than recommended....30
11.1.1 1. A sequence never opens its own port....30
11.1.2 2. Names are declared and resolved up front....30
11.2 Anatomy....30
11.3 The decorator form....31
11.4 A real example....31
12 Context and results....32
12.1 What a sequence is given....32
12.2 What it is deliberately NOT given....32
12.3 Helpers....32
12.4 Passing data between steps....33
13 The runner....34
13.0.1 Figure 3.1: Resolve, then run....34
13.1 Registration....34
13.2 Resolution happens before any traffic....35
13.3 Execution....35
13.3.1 stop_on_fail defaults to True, and that default is load-bearing....35
13.4 The report....35
13.5 Introspection....36
14 Writing a sequence....37
14.1 The skeleton....37
14.2 The file goes in the area’s bin/....37
14.3 Checklist....38

14.4 What does NOT belong in a sequence....38
15 The board registry....39
15.1 BoardSpec....39
15.2 The two boards in this lab....39
15.3 Using it....40
15.4 The CLI....40
16 Programming....41
16.0.1 Figure 4.1: The program path....41
16.1 Order of checks....41
16.2 The environment handed to Tcl....42
16.3 From a Makefile....42
17 Locking and identity....43
17.1 The lock....43
17.2 Why a lock is not sufficient....43
17.3 The identity readback....43
17.4 The record....44
17.5 Status....45
18 Naming, and why the prefixes are load-bearing....46
18.0.1 Figure 5.1: Where each kind of file lives....46
18.1 The table, and the two things that read it....46
18.2 The prefix marks the file, not the directory....47
18.3 Measured status, 2026-09-30....47
19 Resolving paths....49
19.1 What happened....49

19.2 The pattern....49
19.3 Using it....50
19.4 Why an entry point still needs it....50
19.5 The corollary for generators....50
19.6 One name, one thing....50
20 Standing up a new flow....51
20.1 1. Register the board, if it is new....51
20.2 2. Confirm the board is visible before writing any code....51
20.3 3. Create the directory skeleton....51
20.4 4. Add the area’s environment module....51
20.5 5. Give the harness an identity register....51
20.6 6. Write the init sequence first....51
20.7 7. Write the entry point....52
20.8 8. Make it run in simulation....52
20.9 9. Lock the board paths....52
20.10 10. Record what you programmed....52
20.11 Checklist....52

List of Figures

Figure 1.1: Layering of the host-side stack....13
Figure 2.1: The UART flow, host to silicon....19
Figure 3.1: Resolve, then run....34
Figure 4.1: The program path....41
Figure 5.1: Where each kind of file lives....46

List of Tables

No tables in this document.

List of Waveforms

No waveforms in this document.

1 Fpga Systems Mas Index

Generated: 2026-10-13

2 Overview

`projects/fpga-systems/bin/` is the shared host-side layer that every board flow in this repository imports by bare module name. It answers four questions that every flow used to answer for itself:

1. Which serial port is my board on?
2. How do I read and write its registers?
3. What order do my test steps run in, and what did the previous one produce?
4. Which FPGA am I about to program, and is anyone else using it?

Before this layer existed, each flow grew its own answer. Four near-identical `autodetect_port()` functions globbed `/dev/ttyUSB*`; seven copies of `program_fpga.tcl` each hardcoded a JTAG serial and invented their own environment variable to override it. Those copies did not drift because anyone was careless – they drifted because nothing made them one thing.

2.1 What this book covers

The shared layer, the conventions that govern code built on it, and the path from a host program down to a register write landing on silicon.

Chapter	Subject
2	The UART flow: port discovery, probes, the link, the wire protocol
3	Sequences: the model, the context, the runner
4	Boards: the registry, programming, locking, identity
5	Naming and layout conventions, and how paths are resolved
6	Standing up a new flow

2.2 What this book does not cover

- **Method.** How to run a regression, how to write a testbench, how to debug a board. That lives in `vault/handbook/`, which is the repository's memory. Where this book touches a method note it links to it rather than restating it; a second copy is how documentation rots.

- **The RTL on the other side of the UART.** The FPGA-side bridge that turns the ASCII protocol into AXI4-Lite transactions is specified with the RTL.
- **Per-area flows.** What pumice's `seq_char` measures, or what the stream monitor campaign proves, belongs to those areas.

2.3 The one property worth protecting

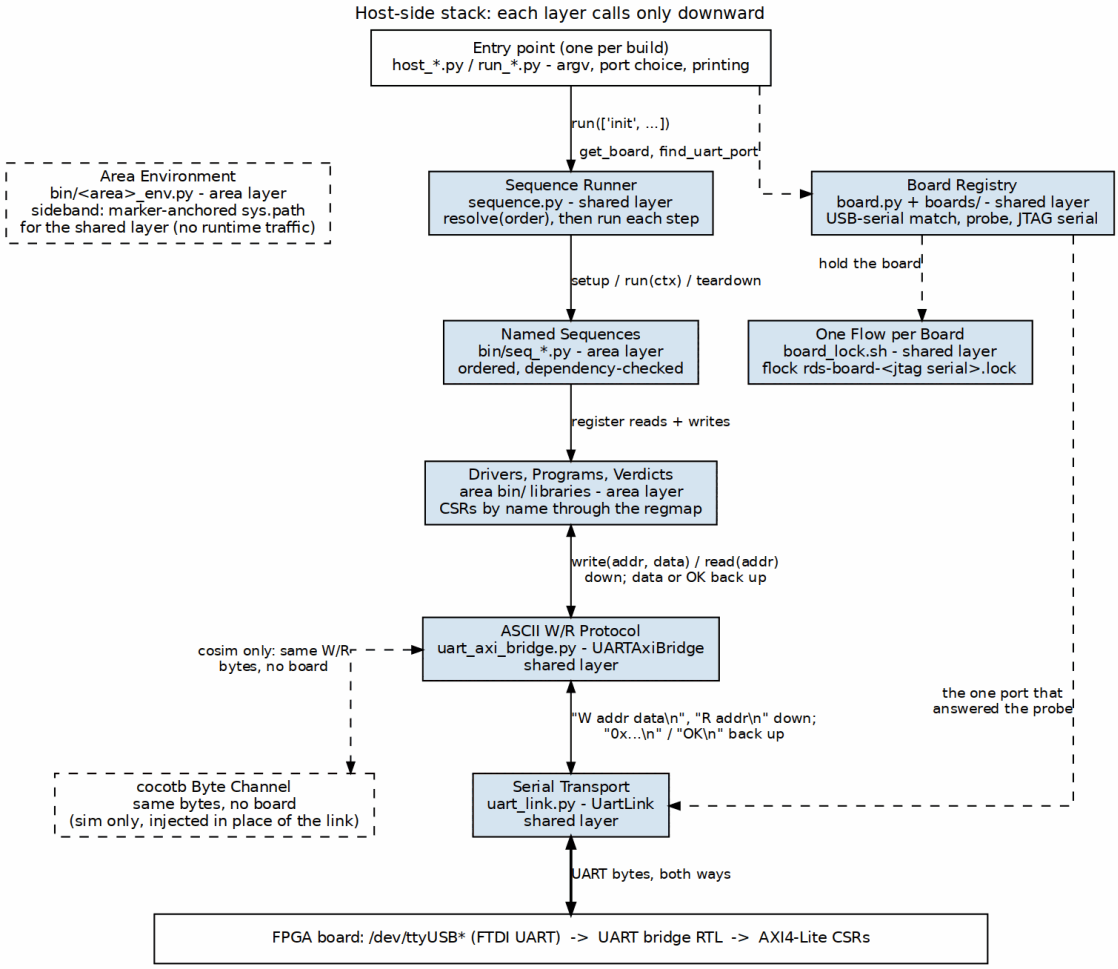
Everything in this layer is arranged so that **the same host code runs against the FPGA and against a simulation**. The byte stream is identical; only the object carrying the bytes differs.

That is not an elegance argument. A harness whose sim path and silicon path are different code has two chances to be wrong and no way to tell which one is. When a board result and a sim result disagree here, the disagreement is about the design, because nothing else differs.

Chapter 2 explains the mechanism (an injected byte channel), and Chapter 3 explains the rule that keeps it true (a sequence never opens its own port).

3 Layering

3.0.1 Figure 1.1: Layering of the host-side stack



Layering of the host-side stack

Four layers, each of which may only reach downward.

3.1 Entry point

One per build: a `host_*.py` or `run_*.py` that parses `argv`, decides which port and which board, constructs the transport once, and prints results. It is the only layer that knows a command line exists.

Entry points are deliberately thin. The work belongs in a sequence or a library, because those are reusable and an entry point is not.

3.2 Area layer

Owned by a unit (pumice, rapids, stream, reed-solomon, cdc). It holds:

- **Sequences** (`seq_*.py`) – named, ordered, dependency-checked steps.
- **Libraries** – drivers, program builders, verdict functions.
- **An environment module** (`<area>_env.py`) – the one place the area records where the shared layers are, described in Chapter 5.

3.3 Shared layer

`projects/fpga-systems/bin/`, specified by this book:

Module	Responsibility
`uart_link.py`	Port enumeration, USB-serial matching, the probe loop, `UartLink`
`uart_axi_bridge.py`	The ASCII W/R protocol over any byte pipe
`sequence.py`	`Sequence`, `SequenceContext`, `SequenceRunner`
`board.py`, `boards/`	The board registry, programming, identity readback
`board_lock.sh`	One flow per physical board
`program_fpga.tcl`, `jtag_readback.tcl`	The two Vivado entry points

It depends on nothing above it, which is what lets a unit be added without touching it.

3.4 Transport and silicon

A `/dev/ttyUSB*` character device on an FTDI part, and the FPGA behind it. The dashed edge in the diagram is the substitution that matters: `UARTAxiBridge` accepts an injected channel, so a cocotb simulation can receive the identical byte stream with no board present.

3.5 The direction rule

A layer may call downward and must not call upward. The one that gets violated in practice is a sequence reaching for a port – which is why the runner enforces it rather than merely recommending it (Chapter 3).

4 Acronyms and terminology

Term	Meaning
AXI4-Lite	The simple AMBA bus the FPGA-side bridge drives. One 32-bit transfer per transaction, no bursts.
BFM	Bus Functional Model. A simulation driver standing in for real traffic.
Byte channel	Any object with `write`, `read_until`, `reset_input_buffer`, `reset_output_buffer`, `close`, `is_open`. A `UartLink` is one; so is a cocotb channel.
CSR	Control and Status Register.
Entry point	A `host_*.py` or `run_*.py`: the file a person or Makefile invokes.
FT2232, FT232R	FTDI USB parts. The FT2232 carries two interfaces (JTAG and UART on one chip); the FT232R carries one.
hw_server	Xilinx's JTAG daemon. Vivado talks to the cable through it.
IDCODE	The JTAG identification register of a device on the chain.
JTAG serial	The USB serial number Vivado uses to pin a `hw_target`.
Probe	A predicate handed an open link that answers "is this the board I want?".
Sequence	A named, dependency-declaring unit of work (Chapter 3).
SequenceContext	What a sequence is given: a bus, a board, parameters, earlier results.
USB serial	The serial number the USB descriptor reports. On a Nexys A7 this is also the JTAG serial.

4.1 Terms used precisely in this book

Port always means a character device path (`/dev/ttyUSB3`), never an RTL port. The RTL sense does not appear in this book.

Board means a `Board` object from the registry – the software handle – unless the text says "physical board".

Sequence means a `Sequence` subclass, not "a sequence of bytes". Where the byte sense is meant the text says "byte stream".

Entry point and **host program** are the same thing. The convention names it by prefix: a file named `host_*` is an entry point. The directory it sits in is not what makes it one (Chapter 5).

5 References

5.1 In this repository

Path	What it is
`projects/fpga-systems/bin/`	The layer this book specifies
`vault/handbook/fpga/cmn-infra/uart-harness.md`	Method: one host program for sim and silicon
`vault/handbook/fpga/cmn-infra/boards.md`	Method: board mechanics and gotchas
`vault/handbook/fpga/cmn-infra/host-stack.md`	Method: the shared py transport stack
`vault/handbook/fpga/cmn-infra/sequences.md`	Method: campaign structure
`vault/handbook/fpga/cmn-infra/flow-layout.md`	**Authority** for the directory skeleton and filename conventions
`vault/handbook/fpga/cmn-infra/flow-migration.md`	Method: migrating a pre-migration `flows-*` flow
`vault/handbook/fpga/cmn-infra/area-structure.md`	Method: where an FPGA project lives
`make/fpga_flow.mk`	The make targets discovered from the filename prefixes
`vault/handbook/dv/registers-by-name.md`	Method: never address a register by offset
`vault/handbook/dv/running-regressions.md`	Method: clean builds, and why a fast pass is fiction
`bin/DOC_GENERATION.md`	How this book becomes a PDF
`bin/TBClasses/harness/`	The simulation-side transport spine the bridge plugs into

The handbook is the authority on method. This book is the authority on the layer. Where they meet, the handbook wins and this book links to it.

5.2 Tracker

Item	Subject
tooling TASK-022	The board lock, the identity readback, and the identity record

Item	Subject
tooling TASK-024	This book

5.3 External

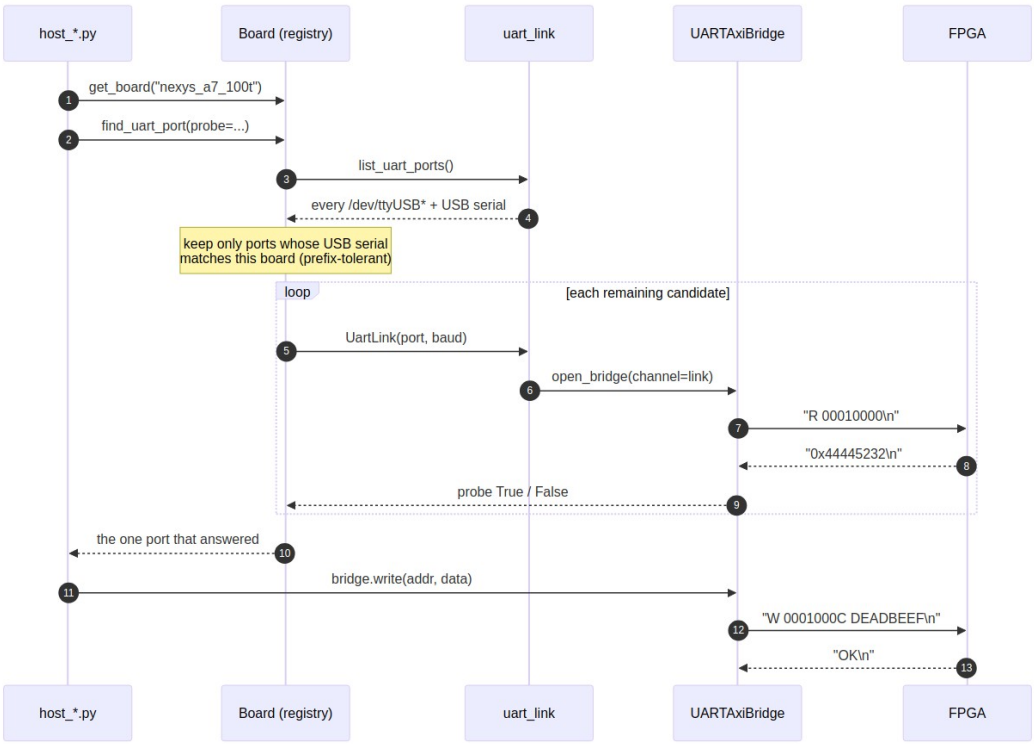
- Xilinx UG908, *Vivado Design Suite User Guide: Programming and Debugging* – `hw_server`, `hw_target`, and the JTAG object model the Tcl scripts use.
- FTDI device datasheets (FT2232H, FT232R) – the USB descriptor fields that carry the serial numbers port matching depends on.
- Digilent board reference manuals (Nexys A7-100T, Genesys 2) – which FTDI part is fitted and how its interfaces are wired.

5.4 A note on register documentation

This book does not list any area's registers. Registers are addressed by name through a generated register map, never by a literal offset, so a register table copied into this book would be a second source that nothing keeps in step. The generated map for an area is the authority; see `vault/handbook/dv/registers-by-name.md` for why this is a rule rather than a preference.

6 Port discovery

6.0.1 Figure 2.1: The UART flow, host to silicon



The UART flow, host to silicon

6.1 The problem

A USB-UART re-enumerates. Unplug a cable, reboot the host, or power-cycle a board and `/dev/ttyUSB3` becomes `/dev/ttyUSB5`. Two boards on one desk makes it worse: the index that worked yesterday now addresses the other board, and a harness that writes to it will be perfectly happy about it.

So a port is never hardcoded, and never merely guessed at by index. It is resolved, every run, by asking which device answers.

6.2 What identifies a board

`list_uart_ports()` enumerates candidates from `serial.tools.list_ports`, falling back to a bare glob of `/dev/ttyUSB*` when pyserial is absent. The fallback matters for board-less callers and minimal environments; the preferred path matters because only pyserial carries the field that identifies the board:

```python
@dataclass(frozen=True)
class UartPort:
    device: str                       # /dev/ttyUSB3
    usb_serial: Optional[str] = None # the field that says WHICH BOARD
    vid: Optional[int] = None
    pid: Optional[int] = None
    description: Optional[str] = None
    manufacturer: Optional[str] = None
    location: Optional[str] = None
```

On a Digilent board with an FT2232, the USB-UART and the JTAG interface are on the **same** part, so the USB serial number and the Vivado JTAG target serial are the same string. That coincidence is what lets `Board.find_uart_ports()` return only the ports belonging to the board you are about to program.

6.3 Matching is prefix-tolerant in both directions

```python
def matches_serial(self, serial): ...
```

An FT2232 exposes one EEPROM serial across its interfaces, and the tooling on either side may or may not append the interface letter – `...D46F` against `...D46FB`. Both strings are normalised (upper-cased, punctuation stripped) and then compared with `startswith` **in both directions**.

An exact-match test here silently finds nothing on some hosts. That failure presents as “board not attached” while the board sits plugged in on the desk, which is precisely the hour this tolerance exists to save.

6.4 The Genesys 2 exception

The Genesys 2 does not share one FTDI part. Its UART is a separate FT232R, so matching UART ports against the JTAG serial finds nothing – the same symptom as above, from a different cause. The registry records the FT232R’s own serial in `uart_serial`, and `BoardSpec.uart_usb_serial` returns it in preference to the JTAG serial:

```python
@property
def uart_usb_serial(self):
    return self.uart_serial or self.jtag_serial
```

Both cables must be plugged in for a UART to exist at all, whatever the bitstream does. See Chapter 4 for the registry, and the `[[boards]]` handbook note for the full gotcha list.

6.5 Listing what is attached

```
python3 projects/fpga-systems/bin/uart_link.py
python3 projects/fpga-systems/bin/uart_link.py --all     # not just
ttyUSB*
```

prints each device with its USB serial and description. This is the first thing to run when a flow says it cannot find its board.

7 Probes

Narrowing by USB serial says which ports belong to a board. It does not say which of them is the one the harness is listening on, nor whether the board is running the bitstream you think it is. A probe answers that.

```
Probe = Callable[[UartLink], bool]
```

A probe is handed an **open** link and returns True or False. It must not raise on a wrong-but-healthy board – returning False is enough, because the loop is going to try the next candidate either way.

The probe stays a caller concern because each harness identifies itself differently. Two shapes occur in this repository, and both ship.

7.1 register_probe – read-only

```
register_probe(addr=0x1_0000, magic=0x44445232)
```

Reads one register and compares it to an identity constant. Used by the rapids (`CSR_ID`), cdc (`BUILD_ID`) and pumice (`BUILD_ID`) harnesses.

Read-only, so it is safe to aim at a board running someone else's bitstream. Prefer this shape when you have the choice.

7.2 scratch_probe – round-trips a scratch register

```
scratch_probe(addr=0x1_0004, magic=0xC0FFEE5A)
```

For harnesses whose identity register is a plain scratchpad rather than a build-ID constant. Used by the stream characterization harness.

This one WRITES, and that makes it dangerous in a way the other is not. Every harness in this lab speaks the same ASCII protocol, so a write lands wherever `addr` points in *that* board's map. It does not bounce off a board that is not yours.

Two mitigations are built in, and neither removes the need to narrow candidates first:

- The original value is read before the write and **restored on a mismatch**, so a probe that hits someone else's board does not abandon a magic number in one of its registers.
- On a match the scratch is zeroed, which is its resting state.

Narrow the candidate set before probing – `Board.find_uart_port(probe=...)` filters by USB serial first – rather than scanning every `ttyUSB` on the host.

7.3 The resolution order

```
find_port(probe=None, want=None, candidates=None, baudrate=...,
timeout=0.4,
          label="harness", pattern="/dev/ttyUSB*", verbose=True) ->
str
```

1. An explicit `want` (the caller's `--port`) is tried **first but still probed**. A stale path in a script therefore fails loudly instead of quietly driving the wrong board. The string `"auto"` means "no preference".
2. Then every remaining candidate, in order.
3. The first that answers wins.

A `probe` of `None` means "take the first port that opens", which is only sensible when exactly one board is attached.

When nothing answers, `find_port` raises `SystemExit` carrying the candidate list and the question worth asking:

```
[autodetect] no <label> responded on any of: ['/dev/ttyUSB0',
'/dev/ttyUSB3'].
Is the board powered and programmed with the right bitstream?
```

`SystemExit` rather than an exception because every caller of this is a CLI, and a traceback helps nobody.

8 The link and the bridge

Two objects sit between a host program and a register: `UartLink` carries bytes, `UARTAxiBridge` turns register operations into bytes. They are separate so that the second can be driven by something other than the first.

8.1 UartLink

A pyserial client that is also a **byte channel**: it satisfies the same protocol as the simulation-side `SerialChannel` in `bin/TBClasses/harness/`.

Method	Purpose
`write(data) -> int`	Send bytes
`read_until(expected=b"\n", size=None) -> bytes`	Read to a delimiter
`reset_input_buffer()`	Discard buffered input
`reset_output_buffer()`	Discard buffered output
`close()`	Close the port
`is_open`	Property

Construction opens the port, sleeps a short settle interval, and flushes both buffers – a USB-UART that has just been opened will otherwise hand you fragments of whatever was in flight before.

```
with UartLink("/dev/ttyUSB3", baudrate=115200) as link:
    bridge = link.bridge()
    bridge.write(0x1_0000, 0xDEADBEEF)
```

`UartLink` is a context manager, and `bridge()` is a convenience for `open_bridge(channel=self)`.

pyserial is imported lazily, inside `__init__` rather than at module scope, so a machine without it can still import `uart_link` for port enumeration and for the unit tests. The sim path never touches a real port.

8.2 UARTAxiBridge

Turns 32-bit register reads and writes into the ASCII protocol of the next chapter.

Method	Returns
`write(addr, data)`	True when the board answered `OK`
`read(addr)`	The 32-bit value, or None on a malformed reply

Method	Returns
`write_verify(addr, data)`	True when a readback matches
`dump_memory(start, num_words=16)`	List of `(addr, data)`
`fill_memory(start, values)`	Writes a list of words

`addr` and `data` accept an `int` or a hex string (`"0x44A00000"`).

8.3 The injection point

```
def __init__(self, port='/dev/ttyUSB1', baudrate=115200, timeout=1.0,
             channel=None):
    if channel is not None:
        self.ser = channel
        return
    import serial
    ...
```

When `channel` is given, no serial port is opened and `port`, `baudrate` and `timeout` are ignored. Anything satisfying the byte-channel protocol works: a `UartLink`, a cocotb driver, or a tracing wrapper that records the wire for later comparison.

This single branch is what makes the sim and silicon paths one piece of code. It is the subject of `05_sim_equivalence.md`.

8.4 Resolving the bridge's location

```
open_bridge(port=None, baudrate=..., timeout=..., channel=None)
```

`uart_axi_bridge` is a sibling module in the same directory, and `open_bridge` inserts that directory on `sys.path` before importing it. The bridge used to live beside the converter RTL it talks to, and every host tool re-derived that location by hand – through `$REPO_ROOT`, or a parent walk whose depth broke the moment the layer moved. There is no path derivation left to get wrong here; see Chapter 5 for the general rule.

9 The wire protocol

Line-oriented ASCII, one request per line, one reply per line. Deliberately trivial: it is readable in a terminal, it costs the FPGA almost nothing to parse, and a human debugging a dead board can type it by hand into `screen`.

9.1 Write

```
host -> board :  "W <addr:08X> <data:08X>\n"
board -> host :  "OK\n"
```

```
cmd = f"W {addr:08X} {data:08X}\n"
self.ser.write(cmd.encode('ascii'))
response = self.ser.read_until(b'\n', size=10)
return response.strip() == b'OK'
```

`write` returns a bool. Anything that is not exactly `OK` is False, including a timeout, which arrives as an empty read.

9.2 Read

```
host -> board :  "R <addr:08X>\n"
board -> host :  "0x<value:X>\n"
```

```
cmd = f"R {addr:08X}\n"
self.ser.write(cmd.encode('ascii'))
response = self.ser.read_until(b'\n', size=20).strip()
if response.startswith(b'0x'):
    return int(response[2:], 16)
return None          # malformed or absent
```

Both fields are fixed-width uppercase hex with no `0x` prefix on the request; the reply to a read **does** carry `0x`. That asymmetry is in the protocol as implemented and is what the parser keys on.

9.3 Defaults

Parameter	Value
Baud	115200 (`DEFAULT_BAUD`)
Data bits	8
Parity	none
Stop bits	1
Flow control	none
Read timeout	1.0 s default, 0.4 s while probing

The baud must match the FPGA's configuration. In simulation it does not: see the next chapter.

9.4 A read of None is ambiguous, and that is worth knowing

`read()` returns None both for “the board sent something I could not parse” and for “the board sent nothing”. Callers that need to distinguish a dead board from a bad address must do so at a higher level – typically by probing a known-good identity register first, which is exactly what the probes in `02_probes.md` are for.

Both failure shapes print the offending response, so the distinction is visible in a log even though it is not in the return value.

9.5 Addressing

Addresses are 32-bit and byte-addressed; a 32-bit register at word index `n` sits at `base + 4*n`. Host code should not compute these by hand: use the generated register map for the area and address registers **by name**. See `vault/handbook/dv/registers-by-name.md` for why offsets written into host code are a recurring source of silent breakage after an RDL change.

10 Sim and silicon equivalence

The property this layer is arranged to protect: **the same host code runs against the FPGA and against a simulation, and the bytes on the wire are identical.**

10.1 The mechanism

One branch, in `UARTAxiBridge.__init__`:

```
if channel is not None:
    self.ser = channel
    return
import serial
self.ser = serial.Serial(port, baudrate, timeout=timeout)
```

On silicon the channel is a `UartLink` wrapping a `/dev/ttyUSB*`. In simulation it is a cocotb UART driver attached to the DUT's RX and TX pins. The protocol code above that branch does not know which it has, because nothing above the branch can tell.

Three channel implementations are in use:

Channel	Where
`UartLink`	Silicon, over pyserial
cocotb UART channel	Simulation, driving the DUT pins
`TracingChannel`	Either, wrapping another channel and recording the wire

`TracingChannel` is what turns "should be identical" into something checkable: run the same host program both ways, compare the recorded byte streams.

10.2 What differs, and what must not

Differs	Must not differ
Baud divider (`CLKS_PER_BIT`)	The byte stream
Wall-clock duration	The register addresses touched
Presence of a board	The order of operations
`ctx.board` (a `Board`, or None)	The host code path

The baud divider is the interesting one. `rs_loop_top.sv` derives the board value from the system clock:

```
localparam int UART_CLKS_PER_BIT = (CFG_SYS_CLK_HZ + CFG_UART_BAUD /
2) / CFG_UART_BAUD;
```

which at 100 MHz and 115200 baud is about 868 clocks per bit. The simulation testbench sets `CLKS_PER_BIT = 4`. Same bytes, two orders of magnitude less simulated time.

10.3 The constraint that follows

A sim-harness test is bounded in simulated time – in this tree, 100 ms. UART traffic at a realistic divider blows that budget immediately, which creates a standing temptation: shorten the campaign so it fits.

That is the wrong lever. Shrinking the campaign changes what is under test, and the shortened version is no longer the thing that runs on the board. Raising the sim baud changes only how fast the identical byte stream is carried.

The reed-solomon harness states this at the point of temptation, in the test that would otherwise be shortened:

```
CLKS_PER_BIT = 4
# ... if the campaign does not fit, the lever is the sim baud
# (CLKS_PER_BIT above), not the parameters and not the campaign.
```

10.4 The anti-pattern

A testbench that builds its own private bridge – speaking the protocol directly to the DUT rather than running the host program – is not an equivalence test. It is a second implementation, and it will agree with the first right up until it matters.

The handbook note `[[uart-harness]]` documents this failure and the flows it has been found in. The rule in Chapter 3 (a sequence never opens its own port) is what keeps the host side substitutable in the first place.

11 The sequence model

A campaign is an **init sequence** followed by one or more **test sequences**. `sequence.py` provides the container; an area provides the sequences.

```
runner = SequenceRunner(ctx)
runner.discover("projects/fpga-systems/NexysA7/mem-ctrl-ip-ddr2/bin")
report = runner.run(["init", "write_read"])
```

11.1 Two rules, enforced rather than recommended

Both of the failure modes these prevent are silent, which is why they are checked in code rather than written in a style guide.

11.1.1 1. A sequence never opens its own port

A sequence is handed a `SequenceContext` carrying an already-built bus. It does not take a `--port`, does not call `find_port`, and does not construct a bridge.

That is what makes the identical sequence run against the FPGA and against a cocotb simulation – only the injected bus differs. A sequence that resolved its own transport would break the equivalence of Chapter 2, which is the one property the whole stack exists to preserve.

11.1.2 2. Names are declared and resolved up front

Every sequence declares a `name`. The runner resolves the entire requested order, plus every `requires`, **before any traffic reaches the board**.

A misspelled name therefore raises immediately instead of skipping a step. This matters more than it sounds: a skipped init looks exactly like a DDR2 timing bug, and costs the same hour to chase.

11.2 Anatomy

```
class Sequence:
    name: str = ""
    description: str = ""
    requires: Seq[str] = ()

    def run(self, ctx) -> Any: ...
    def setup(self, ctx) -> None: ...      # optional, default no-op
    def teardown(self, ctx) -> None: ...   # optional, default no-op
```

The return value of `run` is stored in `ctx.results[name]`, which is how a later sequence consumes an earlier one's output without either knowing the other's internals.

Returning `False` explicitly marks the step failed. Any other value – including None – is a pass.

11.3 The decorator form

For steps needing no setup or teardown:

```
@sequence("write_read", requires=("init",))
def write_read(ctx):
    """One-line description, taken from this docstring."""
    ...
```

This builds the same `Sequence` subclass. The description defaults to the first line of the docstring.

11.4 A real example

```
class Smoke(Sequence):
    name = "smoke"
    requires = ("init",)
    description = "bypass, clean, e = t, e = t + 1"

    def run(self, ctx):
        drv = ctx.bus
        t = ctx.result("init").profile["t"]
        blocks = ctx.param("blocks", 16)
        ...
```

Note what it touches: `ctx.bus` for the device, `ctx.result("init")` for the previous step's output, `ctx.param` for a run-level knob. No port, no baud rate, no board.

12 Context and results

```
@dataclass
class SequenceContext:
    bus: Any = None
    board: Any = None
    params: Dict[str, Any] = field(default_factory=dict)
    results: Dict[str, Any] = field(default_factory=dict)
    log: Optional[Callable[[str], None]] = None
```

12.1 What a sequence is given

Field	What it carries
`bus`	The by-name register interface – a `DeviceBus`, a `Device`, or an area driver. Sequences address registers by name only.
`board`	The `Board` this is running against, or **None in simulation**.
`params`	Run-level knobs from the entry point's command line.
`results`	What earlier sequences returned, keyed by sequence name.
`log`	Where `ctx.say()` writes; defaults to `print`.

12.2 What it is deliberately NOT given

No port. No baud rate. No bitstream path. Transport is already resolved by the time a sequence runs, by the entry point that built the context.

`board` being None in simulation is the tell that a sequence must not depend on it for anything load-bearing. A sequence that needs the board object to work cannot run in sim, and has given up the property of Chapter 2.

12.3 Helpers

```
ctx.say("message")              # to ctx.log, or print
ctx.param("blocks", 16)         # with a default
ctx.result("init")              # RAISES if init has not run
```

`ctx.result` raising rather than returning None is deliberate. Asking for another sequence's result is asserting a dependency; if that dependency did not run, silence would let the sequence proceed on a default and report a number that means nothing. The fix is one word in `requires`.

12.4 Passing data between steps

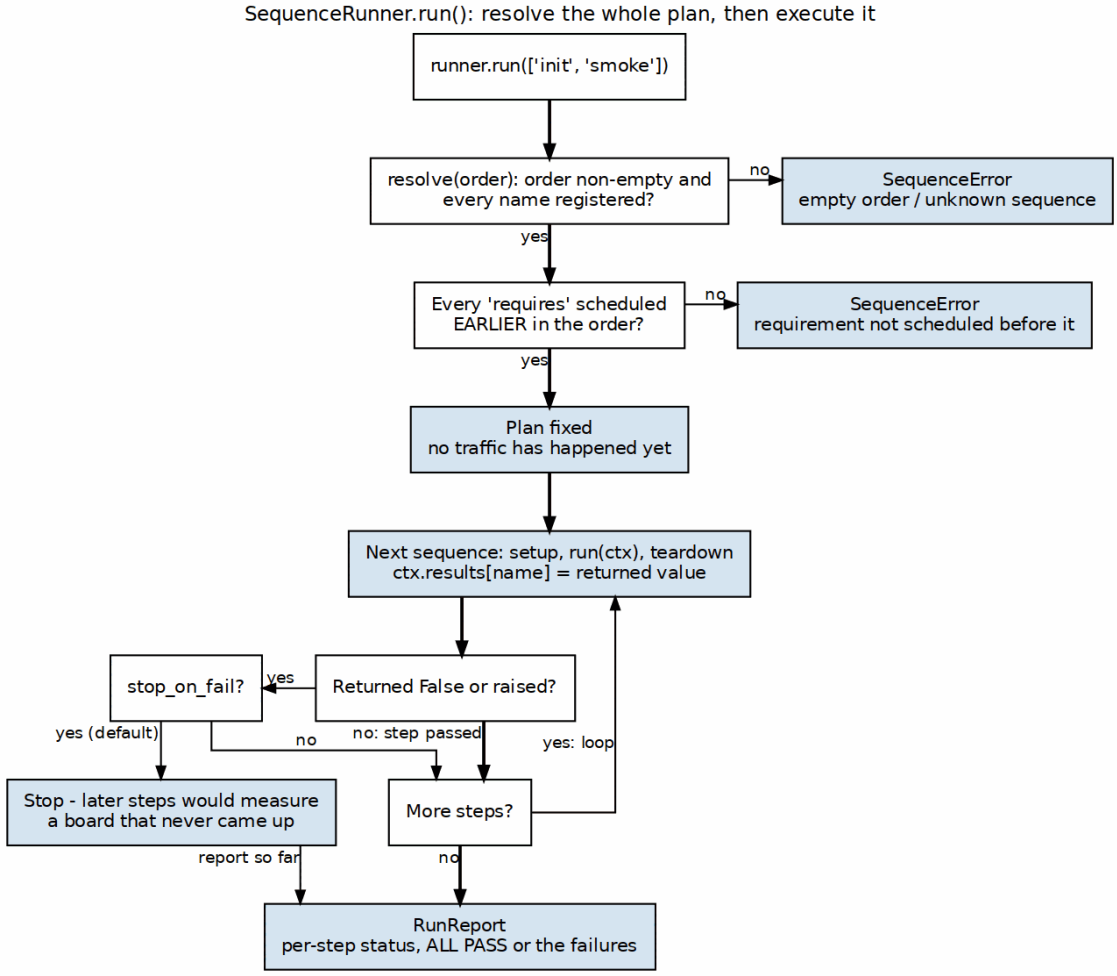
```
class Init(Sequence):
    name = "init"
    def run(self, ctx):
        return {"levelled": True, "taps": taps}

class Measure(Sequence):
    name = "measure"
    requires = ("init",)
    def run(self, ctx):
        taps = ctx.result("init")["taps"]
```

Neither sequence imports the other. The contract is the dictionary and the declared `requires`, which is what lets an area reorder or replace a step without editing its neighbours.

13 The runner

13.0.1 Figure 3.1: Resolve, then run



Resolve, then run

`SequenceRunner` holds the registry for one area and runs a requested order against a context.

13.1 Registration

```
runner.add(SomeSequence)                    # class or instance
runner.discover(path, pattern="seq_*.py")
```

`add` rejects three things outright: an object that is not a `Sequence`, one with no `name`, and a name already registered. The duplicate check names both classes, because two files claiming one name is otherwise found only by noticing that a step ran once when you expected it twice.

`discover` imports every file matching `seq_*.py` in a directory and registers the `Sequence` subclasses it finds. **Discovery is by content, not by filename**: a class in an oddly named file still

registers, and a filename typo shows up at resolve time as a missing name rather than as a silently absent step.

If a `discover` call finds nothing it raises rather than returning an empty registry – an empty registry means nothing would run, which is not a state worth proceeding from.

13.2 Resolution happens before any traffic

```
plan = runner.resolve(["init", "smoke"])
```

Two checks, both before the board is touched:

1. **Every requested name exists.** Unknown names raise `SequenceError` listing what is registered.
2. **Every `requires` is satisfied by something EARLIER in the order.**

The second is checked against what has actually been scheduled, not against the set of names you typed, so `["smoke", "init"]` is an error rather than a coin flip.

Missing dependencies are **reported, never auto-inserted**. Quietly running an init the caller did not ask for is its own surprise, and the fix is one word in the run script.

13.3 Execution

```
report = runner.run(order, stop_on_fail=True)
```

For each sequence: `setup`, `run`, store the result, `teardown`. Teardown runs in a `finally`, so it happens even when `run` raised.

An exception is caught, recorded in the `StepResult` with its traceback logged, and treated as a failure – the run reports rather than masks.

13.3.1 stop_on_fail defaults to True, and that default is load-bearing

These run against hardware. Once init fails, every later step measures a controller that was never brought up. Their numbers are worse than useless, because they look like data: plausible magnitudes, plausible shapes, and entirely meaningless.

```
[run] stopping: later sequences would measure a board that never came
up
```

13.4 The report

```
@dataclass
class StepResult:
    name: str
    ok: bool
    value: Any = None
```

```
    seconds: float = 0.0
    error: Optional[BaseException] = None
```

`RunReport.summary()` prints one line per step with status and duration, and `RunReport.__bool__` is the overall verdict, so `if not report:` works.

Durations are per step. That is not decoration: a step that completes far faster than its known-good time has not done what it claims, which is the standing tell for a stale build or a skipped campaign.

13.5 Introspection

```
runner.names          # sorted registered names
runner.catalog()      # name, description, and requires, one per line
```

`catalog()` is what a `--list` flag should print. It reads the registry, so it cannot drift from what would actually run.

14 Writing a sequence

14.1 The skeleton

```
# <area>/bin/seq_measure.py
from __future__ import annotations

import <area>_env  # noqa: F401   -- side effect: sets sys.path
from sequence import Sequence


class Measure(Sequence):
    name = "measure"
    requires = ("init",)
    description = "sweep the delay line and record the eye"

    def run(self, ctx):
        width = ctx.param("width", 32)
        taps = ctx.result("init")["taps"]
        ctx.say(f"[measure] sweeping {width} points from tap
{taps[0]}")
        ...
        return {"eye": eye}
```

Three things to copy exactly:

1. **`import <area>_env` first**, for its side effect. Chapter 5 explains what it does and why it is not a `sys.path` hack written inline.
2. **`requires` names every earlier step whose result you read.** If you call `ctx.result("init")`, declare it – otherwise the runner cannot tell you scheduled them wrongly, and `ctx.result` will raise at the worst moment instead of at resolve time.
3. **Return something.** It lands in `ctx.results[name]` for later steps. Return `False` only to mean "this step failed".

14.2 The file goes in the area's bin/

`seq_*.py` lives in `<area>/bin/`, not in a build directory, because sequences are shared by every build in that area and by the simulation harness. The prefix is how both `SequenceRunner.discover` and `make` find it – see Chapter 5.

14.3 Checklist

- ☐ Does it touch `ctx.bus` only, never a port or a bridge constructor?

- ☐ Does it work with `ctx.board` as None, so it runs in simulation?
- ☐ Does `requires` list every result it reads?
- ☐ Does it address registers by name rather than by offset?
- ☐ Is the name unique within the area?
- ☐ Does `description` say what it does in one line, for `catalog()`?

The first two are the ones that matter. A sequence failing either has quietly become board-only, and the sim half of the equivalence stops testing anything.

14.4 What does NOT belong in a sequence

Not this	Where it goes
`argparse`	The entry point
Port resolution	The entry point
Board programming	The entry point, or `make program`
Reusable drivers and verdict logic	A library in the area's `bin/`
Printing a final report	The entry point, from the `RunReport`

A sequence that has grown an `ArgumentParser` is an entry point wearing the wrong prefix, and it has lost the ability to be composed with others.

15 The board registry

`boards/` holds one file per board. It is the source of truth for the facts host tooling needs, and it exists because those facts were previously spread across seven copies of a Tcl script, each with its own hardcoded serial and its own environment variable to override it.

15.1 BoardSpec

```
@dataclass
class BoardSpec:
    name: str                                  # registry key
    display_name: str
    part: str                                  # xc7a100tcsg324-1
    jtag_serial: Optional[str] = None          # Vivado hw_target serial
    uart_serial: Optional[str] = None          # only when UART != JTAG
FTDI
    uart_baud: int = DEFAULT_BAUD
    uart_glob: str = "/dev/ttyUSB*"
    notes: Sequence[str] = field(default_factory=tuple)
```

`jtag_serial` pins JTAG when several Digilent boards share a chain and – when `uart_serial` is None – doubles as the USB serial of the UART, because both interfaces are on one FTDI part. `uart_serial` exists for boards where that is not true.

`notes` is not decoration. It carries the board-specific traps that cost an afternoon, and it is printed by `fpga_board.py info`.

15.2 The two boards in this lab

	Nexys A7-100T	Genesys 2
Part	`xc7a100tcsg324-1`	`xc7k325tffg900-2`
JTAG serial	`210292BFA3EE`	`200300B818A0`
UART	Same FT2232HQ as JTAG	**Separate FT232R**, serial `AU05X8RM`
`uart_serial`	None	`AU05X8RM`

They share a JTAG chain, which is why the serial and not the board name is what distinguishes them.

Recorded traps, from the specs themselves:

- **Nexys A7:** Digilent Adept steals the UART ttyUSB; do not power-cycle after programming or the port binding is lost. `ftdi_sio` does not block Vivado programming – no driver dance is needed.

- **Nexys A7:** the lab has had two A7-100T units on this desk. If `make ports` reports nothing, check `udevadm info /dev/ttyUSB*` – a swapped unit presents as exactly that symptom.
- **Genesys 2:** both the FT232R and the JTAG FT2232 must be enumerated or there is no UART, whatever the bitstream does.

15.3 Using it

```
from boards import get_board

b = get_board("nexys_a7_100t")        # or $FPGA_BOARD, else the
default
b.find_uart_ports()                   # only THIS board's ports
port = b.find_uart_port(probe=my_probe)
b.program("bitstream/ddr2_char.bit")
```

Subclass `Board` only when a board needs different behaviour; most need only a `BoardSpec`. `boards/genesys2.py` is a subclass because of its split UART.

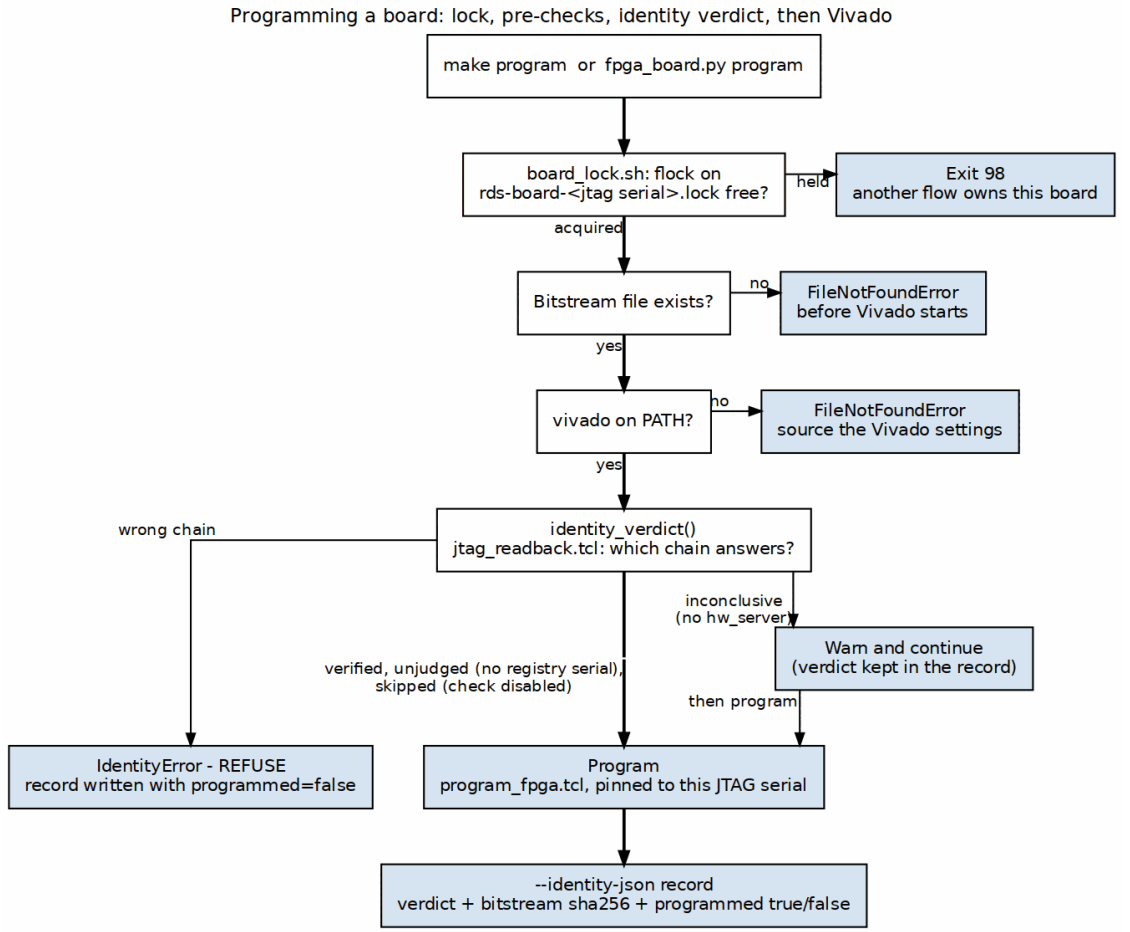
15.4 The CLI

```
fpga_board.py list                          # known boards
fpga_board.py info    --board genesys2      # the spec, including notes
fpga_board.py ports   --board genesys2      # this board's UART ports
fpga_board.py serial  --board genesys2      # the JTAG serial, for Tcl
fpga_board.py readback                      # what is on the JTAG chain
fpga_board.py program --bitstream x.bit
```

`serial` prints nothing and exits 0 when the board has no serial, so a caller can substitute it unconditionally and get “any target” rather than the literal string `None` used as a match pattern.

16 Programming

16.0.1 Figure 4.1: The program path



The program path

```
fpga_board.py program --board nexys_a7_100t --bitstream build/top.bit
```

replaces each flow's own `vivado -source tcl/program_fpga.tcl` recipe. The board facts come from the registry, so a flow's Makefile carries neither a serial nor its own copy of the Tcl.

16.1 Order of checks

Cheap and local first, because a thirty-second Vivado startup to be told the file is missing is pure waste:

1. **Bitstream exists** – `FileNotFoundError` naming the path, suggesting `make bitstream`.
2. **`--dry-run`** – prints the environment and the command, runs nothing.

3. **Vivado on PATH** – `FileNotFoundError` suggesting the settings script.
4. **Identity** – the subject of `03_locking_and_identity.md`.
5. **Program** – `program_fpga.tcl`, pinned to this board's JTAG serial.

16.2 The environment handed to Tcl

Variable	Meaning
`FPGA_BITSTREAM`	Absolute path to the `.bit`
`FPGA_JTAG_SERIAL`	The serial to pin, absent for "any target"
`FPGA_BOARD`	The registry key

A single parameterised Tcl reading these replaces the per-flow copies.

16.3 From a Makefile

`make/fpga_board.mk` wraps the CLI:

```
BOARD_LOCK   ?= $(FPGA_BIN)/board_lock.sh
BOARD_LOCKED  = $(BOARD_LOCK) --board $(BOARD) --
```

and the `program` recipe runs under `$(BOARD_LOCKED)`. A flow gets locking by using the standard recipe rather than by remembering to.

17 Locking and identity

Two mechanisms, and they are complements rather than alternatives: **a lock prevents a collision, a readback detects one that happened anyway.**

17.1 The lock

`board_lock.sh` is a command prefix:

```
board_lock.sh --board nexys_a7_100t -- make program
```

It resolves the board's JTAG serial through `fpga_board.py serial`, falls back to the board name when there is none, sanitises the result, and takes a non-blocking `flock` on `$RDS_BOARD_LOCK_DIR/rds-board-<key>.lock` (default `/tmp`). On conflict it prints a hint and exits **98**; otherwise it `exec`s the payload.

Keyed on the JTAG serial, not the build directory. Two different build directories drive the same physical board, so a build-keyed lock lets them collide while reporting success. Two boards sharing one JTAG chain still get separate keys, because their serials differ.

The lock is held for the life of the payload: `exec 9>` the lock file, `flock -n 9`, then `exec` the command. The descriptor survives `exec`, the lock lives on the open file description, and the kernel releases it when the process dies – including when it is killed.

A consequence worth knowing: a Python `BoardLock` nested inside a shell-locked payload would be refused by the kernel, since the tree already holds the lock. The Python side therefore detects an inherited descriptor and re-locks to prove ownership; an adopted descriptor is never closed on exit.

17.2 Why a lock is not sufficient

A harness records the sha256 of the bitstream **it** programmed. That describes the file it sent, not the device that received it. If a board is reprogrammed or re-enumerates mid-run, the results file carries no error, no timeout and no mismatch – it is simply someone else's measurements under your name.

`fuser /dev/ttyUSB*` does not close this gap either: it sees only the UART half of a board. Measured on this bench, the Genesys 2's JTAG serial appears on **zero** tty nodes.

17.3 The identity readback

`jtag_readback.tcl` lists the chain read-only. `parse_readback` turns its output into targets and devices, and is pure, so it is unit-testable without Vivado and without a board.

`identity_verdict()` returns the check as data and never raises:

Status	Meaning	Effect on `program`
`verified`	This board is on the chain with a device behind it	Proceed
`wrong`	The chain does not hold this board	**Refuse**
`inconclusive`	The chain could not be read	Warn, proceed
`unjudged`	This board has no registry serial	Proceed
`skipped`	`--no-verify-identity` was passed	Proceed

The asymmetry is deliberate. A proven wrong chain refuses. An inconclusive readback only warns, because refusing when `hw_server` is unreachable would break every flow where programming works but the daemon is not running – and this sits on thirteen board paths at once. A board with no registry serial is returned `unjudged` rather than passed; claiming success there would be a checker that cannot fail.

A wedged `hw_server` is `inconclusive`, not an exception: the readback carries a 240 s timeout, and letting `TimeoutExpired` escape would take `program` down, converting the warn-only path into the refusal the asymmetry exists to avoid.

17.4 The record

A warning printed to a terminal is indistinguishable from a pass once it scrolls. So the verdict is written beside the evidence:

```
fpga_board.py program --bitstream top.bit --identity-json
run/identity.json
```

```
{
  "when": "2026-09-30T14:22:05-0700",
  "board": "nexys_a7_100t",
  "expected_serial": "210292BFA3EE",
  "bitstream": "/abs/path/top.bit",
  "bitstream_sha256": "...",
  "programmed": true,
  "identity": {"status": "verified", "detail": "", "chain": {...}}
}
```

Two fields exist for reasons that are easy to miss:

- **`programmed`** – the record is written on the refusal path too, where the sha256 describes a bitstream that never reached the device. Without this flag the file reads as “this bitstream is on that board”, which is exactly the false claim the mechanism exists to prevent. It is also false on a non-zero Vivado exit.
- **`identity.status` including `skipped`** – `--no-verify-identity` still writes the file. A missing file is not a statement; it is an absence a harness tolerates without noticing.

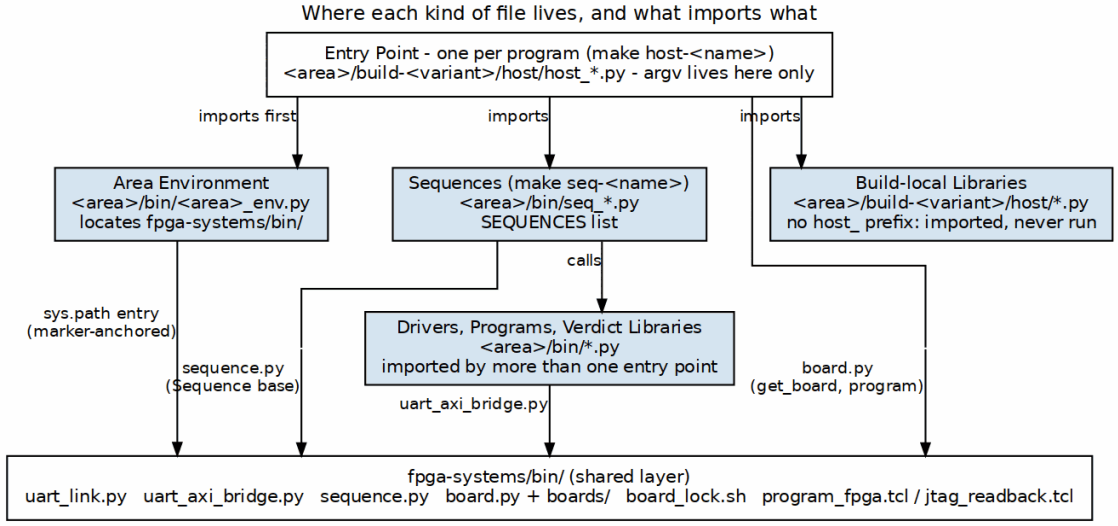
Without this record a run that could not look writes exactly what a run that looked and approved writes, and six weeks later nobody can tell which one they are holding.

17.5 Status

The hardware path is **unexercised**. The parser and the decision logic are covered by hardware-free tests, deliberately, because opening the hardware manager to test this would violate the very lock discipline it accompanies. The first real `make program` on a free board remains the outstanding verification; see tooling TASK-022.

18 Naming, and why the prefixes are load-bearing

18.0.1 Figure 5.1: Where each kind of file lives



Where each kind of file lives

The authority for the directory skeleton and the filename conventions is the handbook note `[[flow-layout]]`, not this chapter. This chapter records only what binds those conventions to the layer specified in this book: the prefixes are not a style preference, they are how two separate mechanisms *find* files.

18.1 The table, and the two things that read it

Pattern	Role	Discovered as
`bin/run_*.py`	A runner: drives a whole plan	`make run-<name>`
`bin/seq_*.py`	One sequence	`make seq-<name>`
`host/host_*.py`	A standalone host program	`make host-<name>`
`host/test_*.py`	pytest	Via `SIM_TESTS`, never a make target
`host/<other>.py`	A library: imported, never run	Nothing
`fpga/tcl/*.tcl`	A Vivado script	`make tcl-<name>`

Two mechanisms depend on this, and both are plain globs:

```
# make/fpga_flow.mk
RUN_SCRIPTS   := $(sort $(wildcard $(SEQ_DIR)/run_*.py))
SEQ_SCRIPTS   := $(sort $(wildcard $(SEQ_DIR)/seq_*.py))
HOST_PROGRAMS := $(sort $(wildcard $(HOST_DIR)/host_*.py))
```

```
# sequence.py
def discover(self, path, pattern: str = "seq_*.py"): ...
```

So a misnamed file is not merely untidy – **it is invisible**. A sequence not matching `seq_*.py` is never registered; a host program not matching `host_*.py` never becomes a make target. `make targets` prints exactly what was found on disk, so "what can I run here" is never a guess, but it can only print what the glob matched.

18.2 The prefix marks the file, not the directory

`host_*` identifies an **entry point**. It does not mean "everything in `host/` is a program", and the distinction is load-bearing in both directions:

- `build-loop/host/` holds `host_rs_loop.py` (the entry point, carrying the `ArgumentParser`) beside `rs_loop.py` and `rs_loop_programs.py`. Those two parse no arguments and are correctly libraries that happen to sit next to the entry point using them.
- Four stream entry points are 21 to 33 line shims that bootstrap `sys.path` and call `main()` from a same-named library in the area's `bin/`. The entry point is the shim; the `ArgumentParser` is in the library.

`host_bus_meters.py` states the rule at the point where someone would get it wrong:

> The implementation is `bin/bus_meters.py`, at COMPONENT level because it is a LIBRARY as well as a program – `host_ext_char` and the cosim tests import its readers. **Entry points are `host_*` and live in a build; anything imported by more than one of them lives in `bin/`.**

Renaming a library to `host_*` is therefore wrong. If a file is imported by anything, it is a library that happens to be runnable, not a program. `[[flow-layout]]` records why discovery uses the prefix rather than scanning for a `__main__` guard: the guard made the role invisible in `ls` and swept up library modules carrying a debug CLI.

18.3 Measured status, 2026-09-30

Nine build directories follow the skeleton (`build-<name>/`). Two do not: `Genesys2/rapids/flows-rapids/` and `Genesys2/rapids_beats/flows-rapids-beats/`.

Those two are **pre-migration flows, not a competing convention**. `flows-*/` is the layout `[[flow-migration]]` names as the thing being migrated away from, and neither includes

`make/fpga_flow.mk` at all – so none of the discovery above applies to them. Their `run_*.py` sitting in `host/` rather than `bin/` is a consequence of that, not a second opinion about where runners go.

The practical reading: there is one convention, it is documented in two places that agree, and the exceptions are a tracked migration backlog.

19 Resolving paths

Anchor to a marker; never count directory levels. This has a date and a cost attached, recorded in `[[flow-layout]]`.

19.1 What happened

The shared board and UART layer moved from `fpga/bin` to `projects/fpga-systems/bin`, and every hand-counted path broke at once:

- `pumice_env.py` counted five levels to the repo root.
- `ddr2_char.py` carried its own copy of the same walk.
- `uart_link.py` counted three and silently resolved to `<root>/projects/projects/...`.
- A test asserted the literal string `fpga/bin/program_fpga.tcl`.

Four independent copies of one fact, each broken separately, none of which told you what was wrong – a wrong path produces an `ImportError` naming a module, not a layout.

19.2 The pattern

Each area keeps exactly one module that knows where the shared layer is. It searches **upward for a marker file**, honouring `REPO_ROOT` first:

```
_MARKER = os.path.join("projects", "fpga-systems", "bin",
"uart_link.py")

def _repo_root() -> str:
    env = os.environ.get("REPO_ROOT")
    if env and os.path.isfile(os.path.join(env, _MARKER)):
        return env
    here = os.path.dirname(os.path.abspath(__file__))
    while True:
        if os.path.isfile(os.path.join(here, _MARKER)):
            return here
        parent = os.path.dirname(here)
        if parent == here:
            raise RuntimeError("cannot find the repo root")
        here = parent
```

then puts the shared directory, and any area-local directory, on `sys.path`:

```
FPGA_BIN = os.path.join(REPO_ROOT, "projects", "fpga-systems", "bin")
for p in (FPGA_BIN, HOST_DIR):
    if p not in sys.path:
        sys.path.insert(0, p)
```

Cost is one `os.path.isfile` per level. Benefit is that the next move breaks nothing.

19.3 Using it

Import it first, for its side effect, before importing anything shared:

```
import rs_env  # noqa: F401
from sequence import Sequence
from boards import get_board
```

The `noqa` is load-bearing documentation: a linter would otherwise remove an import whose only purpose is the side effect, and the file would fail at the next line.

19.4 Why an entry point still needs it

`env_python` puts `projects/fpga-systems/bin` on `PYTHONPATH`, so `sequence`, `boards` and `uart_link` import by bare name in a properly configured shell. That does **not** cover the area's own `bin/`, where its drivers and sequences live. A host program under `build-<variant>/host/` importing an area library therefore still needs the environment module, and omitting it produces an `ImportError` on a first board run – found exactly that way in the reed-solomon flow.

If the chapter's skeleton (Chapter 6) looks like it has a redundant import at the top, that is the one.

19.5 The corollary for generators

A tool that writes output must anchor its default output path to **its own location**, not the current directory. `elaborate_a7ddrphy.py` defaulted to a bare relative path and, run from the wrong directory, produced `rtl-vivado/rtl-vivado/a7ddrphy/`.

19.6 One name, one thing

Two files called `run_smoke.py` once lived in one flow – the sequence runner in `bin/` and an older end-to-end program in `host/`. Different code, same name, and `make run-smoke` could only ever mean one of them.

20 Standing up a new flow

The order below is the one that fails earliest. Each step is checkable before the next becomes possible.

20.1 1. Register the board, if it is new

Add `boards/<name>.py` with a `BoardSpec`. The fields that matter are `part`, `jtag_serial`, and – only when the UART is not on the JTAG FTDI – `uart_serial`. Put every board-specific trap in `notes`; that is what `fpga_board.py info` prints.

```
python3 projects/fpga-systems/bin/fpga_board.py list
python3 projects/fpga-systems/bin/fpga_board.py info --board <name>
```

20.2 2. Confirm the board is visible before writing any code

```
python3 projects/fpga-systems/bin/uart_link.py        # ports and USB
serials
python3 projects/fpga-systems/bin/fpga_board.py ports --board <name>
python3 projects/fpga-systems/bin/fpga_board.py readback
```

If `ports` prints nothing, the problem is the registry's serial or the cabling, not your code. On a Genesys 2 both cables must be enumerated.

20.3 3. Create the directory skeleton

Follow `[[flow-layout]]`; it is the authority. The shape is `<area>/bin/` for sequences and area libraries, and `<area>/build-<name>/` per bitstream. Use `build-`, not `flows-` – see Chapter 5.

20.4 4. Add the area's environment module

`<area>/bin/<area>_env.py`, anchored to the marker as in Chapter 5. Write this before the first sequence; everything imports it.

20.5 5. Give the harness an identity register

Pick a probe shape (Chapter 2) and make the FPGA answer it. Prefer a read-only build-ID constant over a writable scratch register: `register_probe` is safe to aim at a board running someone else's bitstream and `scratch_probe` is not.

20.6 6. Write the init sequence first

`<area>/bin/seq_init.py`. Everything else will declare `requires = ("init",)`. It returns what later steps need – taps, a profile, a topology.

Read the build's topology from a register rather than assuming it. A single-decoder build once failed every run because the host was faithfully scoring a checker that was tied off. The host should ask the hardware what it is, not be told at the command line.

20.7 7. Write the entry point

`<area>/build-<name>/host/host_<thing>.py`. It parses `argv`, resolves the board and port, builds the bus, constructs a `SequenceContext`, discovers the area's `bin/`, and calls `runner.run([...])`. It should be thin.

20.8 8. Make it run in simulation

This is the step that is easiest to skip and most expensive to add later. Drive the same host code through a cocotb byte channel with `ctx.board = None`. If a sequence resolves its own port or needs the board object, it will fail here – which is the point.

Raise the sim baud (`CLKS_PER_BIT`) to fit the time budget. Do not shrink the campaign; the shortened version is no longer what runs on the board.

20.9 9. Lock the board paths

Use the standard `program` recipe so `board_lock.sh` wraps it. A flow that invokes Vivado directly gets no lock, and nothing will tell it so.

20.10 10. Record what you programmed

Pass `--identity-json` from any flow whose results are evidence, so the identity verdict lands beside the bitstream sha256.

20.11 Checklist

- ☐ `fpga_board.py info`, `ports`, `readback` all answer
- ☐ An environment module anchored to the marker, not to a level count
- ☐ An identity register, read-only if possible
- ☐ `seq_init.py` returning what later steps need
- ☐ Sequences touch `ctx.bus` only
- ☐ The same host code runs in sim with `ctx.board = None`
- ☐ Board-touching make targets run under the lock
- ☐ `--identity-json` wherever results are kept